



MIVA
OPEN UNIVERSITY

LAB - COS 202

COMPUTER PROGRAMMING II OUTLINE

MIVA OPEN UNIVERSITY

HEADQUARTERS

PLOT 1059, O.P. FINGESI ROAD,

UTAKO, ABUJA

COURSE DEVELOPMENT TEAM

Instructional Designer: Abdullahi Abubakar Kawu

Language Editor: Udochi Obiukwu, MA English Language

Content Editor: Prof Sylvanus Ehikioya

Table of Contents

Lab - Cos 202 - Computer Programming Ii Outline	6
Learning Outcomes	7
LAB 1: Introduction to C Programming and Development Environment Setup	7
Experiment 1.1: Setting up the JetBrains Clion environment for C++.	9
Exercise 1.1: Installation and Configuration of JetBrains Clion environment.	13
Experiment 1.2 Print a simple statement: “This is C++ programming”	14
Exercise 1.2: Modify the program to print a personalized greeting.	16
Experiment 1.3: Write a simple C++ program to create Variables	16
Exercise 1.3: Create a variable to store float data type number and integer data type number and print the output.	18
Experiment 1.4: Write a C++ program to perform Arithmetic Operations (Addition, Subtraction, Multiplication and Division) accepting 2 inputs from the user.	18
Exercise 1.4: Write a C++ program to perform Arithmetic Operations (Addition, Subtraction, Multiplication and Division) accepting 5 inputs from the user	21
LAB 2: Control Structures and Functions	22
Experiment 2.1: Conditional Statements (if, if else)	22
Exercise 2.1: Write a program that determines whether a given number is even or odd	27
Experiment 2.2: Loops (while, for)	27
Exercise 2.2: Write a program using a loop to calculate the sum of first n natural numbers.	32
Experiment 2.3: Functions, Parameters, and Return Values	32
Exercise 2.3: Implement a guessing game where the user has to guess a random number within a specified range	38
LAB 3: Arrays and Strings	39
Experiment 3.1 Working with One-Dimensional Arrays	40
Exercise 3.1: : Write a program to find the sum and average of elements in an array.	41
Experiment 3.2: Multidimensional Arrays	42
Exercise 3.2: Implement a program that finds the largest and smallest values in an array.	44
Experiment 3.3: String Manipulation and C-Style Strings.	44
Exercise 3.3: Implement a program that finds the largest and smallest values in an array.	46
LAB 4: Object-Oriented Programming Basics	47
Experiment 4.1: Introduction to Classes and Objects	47
Exercise 4.1: Create a class representing a student with attributes like name, roll number, and grade. Instantiate and display student objects.	49
Experiment 4.2: Constructors and Destructors	50
Experiment 4.3: Member Functions and Data Members	53
Exercise 4.3: Design a class for a simple bank account with constructors to initialize account details.	56
LAB 5: Advanced OOP Concepts	57

Experiment 5.1: Inheritance and Polymorphism	57
Exercise 5.1: Create a base class "Shape" and derive classes for different shapes (e.g., circle, rectangle). Calculate and display their areas using polymorphism.	63
Experiment 5.2: Encapsulation and Abstraction	64
Exercise 5.2: Implement a basic employee hierarchy with a base class and derived classes for various employee types.	70
LAB 6: Pointers and Dynamic Memory Allocation	71
Experiment 6.1: Pointers and References	71
Exercise 6.1: Write a program that dynamically allocates an array of integers of a user-specified size using the 'new' keyword. Populate the array with user input and then display the sum of the elements. Don't forget to use 'delete' to free the allocated memory.	75
Experiment 6.2: Dynamic Memory Allocation (new and delete)	75
Exercise 6.2: Implement a dynamic stack data structure using dynamic memory allocation (new and delete).	79
Experiment 6.3: Working with Pointers and Objects.	80
Exercise 6.3: Create a class representing a simple "Student" with attributes	83
LAB 7: File Handling and Exception Handling	84
Experiment 7.1: Reading and Writing Files	84
Exercise 7.1: Write a program that prompts the user to enter two numbers and performs division. Implement exception handling to catch and handle any division by zero exception. Display an error message if such an exception occurs.	87
Experiment 7.2: Exception Handling (try, catch, throw)	87
Exercise 7.2: Extend Exercise 7.1 by adding exception handling for invalid input	89
LAB 8: Standard Template Library (STL)	91
Experiment 8.1: Introduction to STL Containers (Vectors, Lists)	91
Exercise 8.1: Create a program that uses a vector to maintain a list of student names. Allow the user to add new students to the vector and display the list of students. Implement a function to remove a student by name from the vector.	98
Experiment 8.2: STL Algorithms (Sorting, Searching)	98
Exercise 8.2: Write a program that uses the STL algorithm 'binary_search' to search for a specific element in a sorted vector of integers. Prompt the user to enter a number, populate and sort the vector, and then search for the entered number using 'binary_search'. Display whether the number was found or not.	101
LAB 9: Final Project (CA) - 2 weeks	101

Lab - Cos 202 - Computer Programming Ii Outline

Learning Outcomes

At the end of the course, students should be able to

1. develop solutions for a range of problems using object-oriented programming in C++, use modules/packages/namespaces for programme organization,
2. use API in writing applications,
3. apply divide and conquer strategy to searching and sorting problems using iterative and/or recursive solutions and explain the concept of exceptions in programming and how to handle exceptions in programmes.
4. write simple multithreaded applications and design and implement simple GUI applications.

Lab work: Programming assignments leading to extensive practice in problem-solving and programme development with emphasis on object orientation. Solving basic problems using static and dynamic data structures. Solving various searching and sorting algorithms using iterative and recursive approaches. GUI programming.

LAB 1: Introduction to C Programming and Development Environment

Setup

Objectives

- To be familiar with the Programming Environment
- To have a good understanding of the syntax and structure of C++ programming
- To understand the basics of C++

Experiment 1.1: Setting up the JetBrains Clion environment for C++.

To efficiently run any software, you should understand the software and hardware requirements. There are different types of Integrated Development Environment (IDE) software used for programming. For the purpose of this Course, we will be using JetBrains Clion which supports the GNU C/C++ Compiler. However, others exist such as GCC / Turbo C/C++. The GNU C/C++ compiler is the most popular and freely available compiler.

System requirements for JetBrains Clion:

- 64-bit version of Windows 10 1809 and later, or Windows Server 2019 and later
- 2 GB free RAM minimum, 8 GB of total system RAM recommended
- 3.5 GB hard disk space, SSD recommended
- 1024x768 minimum screen resolution

1. Visit <https://www.jetbrains.com/clion>
2. Click on **Download button** on the Menu to Visit the relevant Installer based on your Operating System (OS) or visit <https://www.jetbrains.com/clion/download> and select your OS
3. Click on Download and wait for the download to complete
4. After Download is completed, follow the on screen instructions to install the IDE on your PC/machine. If you're using Windows, you should see a screen similar to Figure 1 below. Click on Next till you finish the installation.

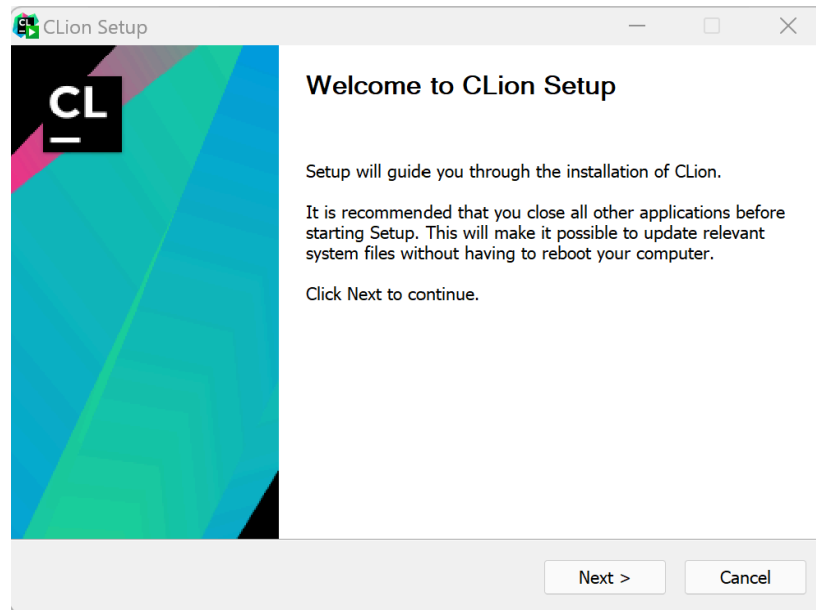


Figure 1: CLion Setup For Windows

5. After the installation is completed, you should see a screen similar to this Figure 2 below:

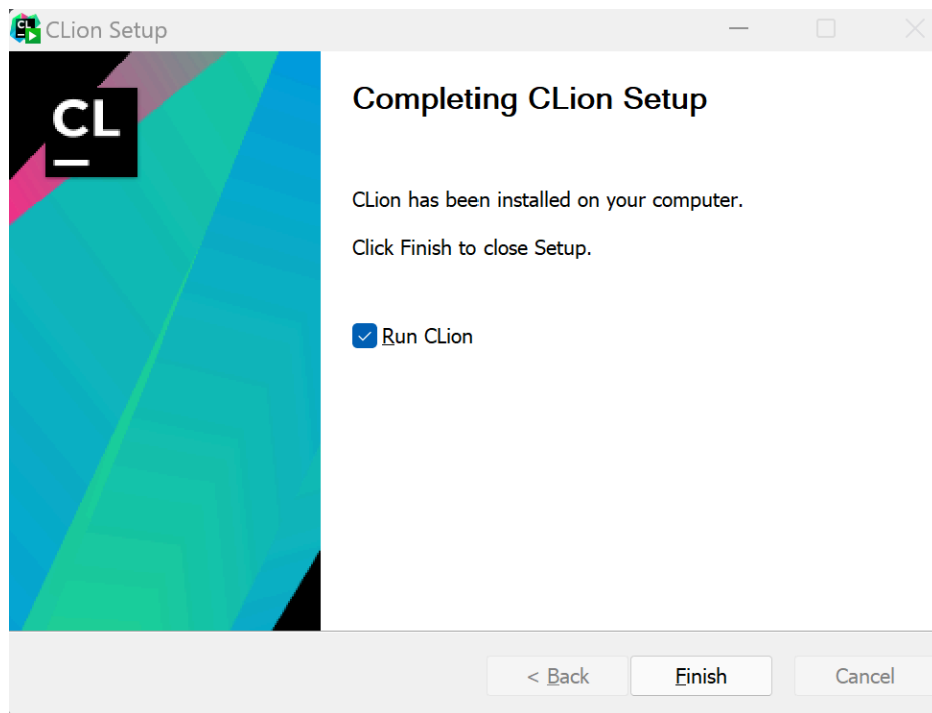


Figure 2: Setup Completed

6. **Launch and Configure CLion:**

- Once installed, launch CLion.
- During the first startup, CLion will ask you to configure your development environment settings (like UI theme, etc.)
- Set up your toolchain (like MinGW or other suitable compiler and debugger). For the purpose of this Lab, we will install/configure the MinGW toolchain.

Note, you can find alternative installations for other toolchain here:
<https://www.jetbrains.com/help/clion/quick-tutorial-on-configuring-clion-on-windows.html>

Post-Installation

- Licensing
- After installation, you will need to activate CLion. As a Miva Student, you will be given a licence to activate the product with a JetBrains Account.
- Toolchains and Compilers:

Note: CLion does not come with a compiler or debugger - you will need to install these separately and configure them in CLion. Instructions for doing this are available in the CLion documentation. Also, always refer to the [official CLion

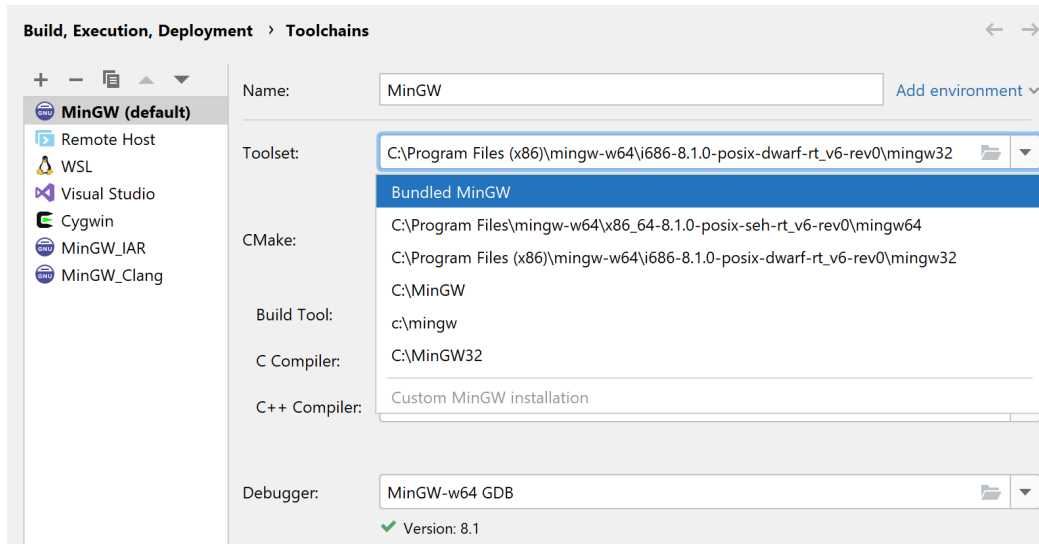
documentation] (<https://www.jetbrains.com/clion/documentation/documenta>

7. Configure a MinGW toolchain

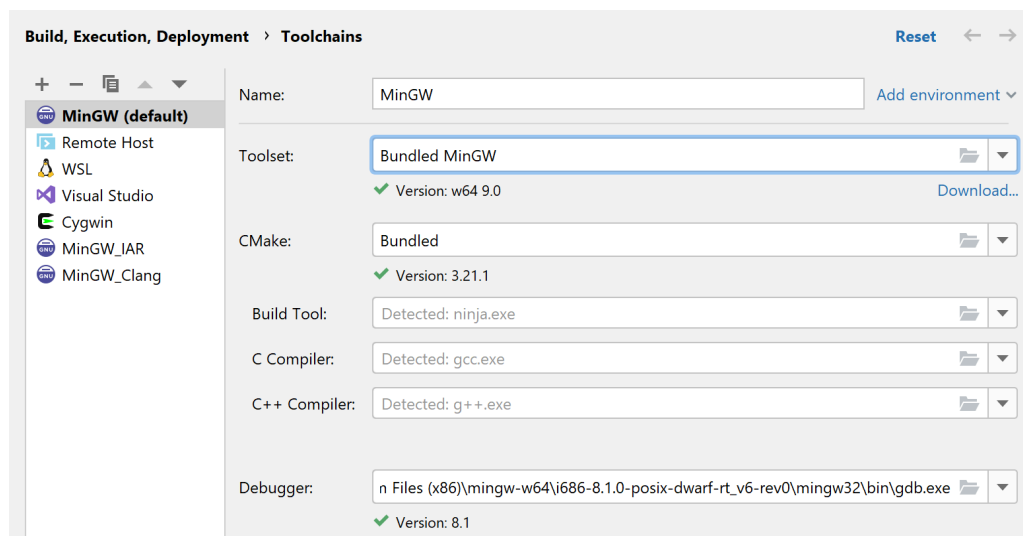
Go to File | Settings | Build, Execution, Deployment | Toolchains.

Click + and select MinGW to add a new MinGW toolchain.

In the Toolset field, you will see Bundled MinGW, which is the default option. If required, open the field to select from the list of other available installations:



Wait until the tools detection finishes.



Select the Debugger: you can use either bundled GDB, your MinGW GDB, or a custom GDB binary. Bundled GDB is recommended, since it is guaranteed to include Python support required for CLion data renderers. GDB does not yet support debugging on Windows ARM64. For this type of system, CLion's bundled GDB is intended to be used for remote debugging only. If required, specify the script to initialize the environment.

Click Apply when all the tools are set correctly.

When using a custom MinGW installation, if CLion cannot detect the compilers, double-check the installed packages in MinGW Installation Manager.

Exercise 1.1: Installation and Configuration of JetBrains CLion environment.

Students are to Setup their PCs with the JetBrains CLion IDE.

Instructions:

- As demonstrated in the experiment (refer back to C programming Manual), students complete their installations and configuration.

Further Activity: Students to Create a New Project Named

MyFirstCPPProgram, then click on Run

Experiment 1.2 Print a simple statement: "This is C++ programming"

Algorithm:

- Start
- Print Message
 - Output the string "This is C++ Programming" to the console.
- End

Code Snippet

```
cpp Copy code  
  
#include <iostream>  
using namespace std;  
  
int main() {  
    // Prints the string enclosed in double quotes  
    cout << "This is C++ Programming";  
    return 0;  
}
```

Output

```
This is C++ Programming
```

Summary:

When this C++ program is run:

- It starts executing from the main function.
- It uses cout to print "This is C++ Programming" to the console.
- Finally, it returns 0, signaling to the operating system that it has executed successfully.

Exercise 1.2: Modify the program to print a personalized greeting.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable

Experiment 1.3: Write a simple C++ program to create Variables

Algorithm:

1. Start
2. Declare and Initialize Variables
 - Declare an integer variable, assign it a value of 10.
 - Declare a float variable, assign it a value of 3.14.
 - Declare a character variable, assign it the value 'A'.
 - Declare a boolean variable, assign it the value true.
 - Declare a double variable, assign it a value of 3.141592653589793.
3. Display Values
 - Print the value of the integer variable to the console with a descriptive message.
 - Print the value of the float variable to the console with a descriptive message.
 - Print the value of the character variable to the console with a descriptive message.
 - Print the value of the boolean variable to the console with a descriptive message.

- Print the value of the double variable to the console with a descriptive message.

End

```
cpp Copy code

#include <iostream>
using namespace std;

int main() {
    // Declaring an integer variable and initializing it with the value 10
    int integerVar = 10;

    // Declaring a floating-point variable and initializing it with the value
    float floatVar = 3.14f;

    // Declaring a character variable and initializing it with the character
    char charVar = 'A';

    // Declaring a boolean variable and initializing it with the value true
    bool boolVar = true;

    // Declaring a double precision floating-point variable and initializing
    double doubleVar = 3.141592653589793;

    // Printing the variables to the console
    cout << "Integer Variable: " << integerVar << endl;
    cout << "Float Variable: " << floatVar << endl;
    cout << "Character Variable: " << charVar << endl;
    cout << "Boolean Variable: " << boolVar << endl;
    cout << "Double Variable: " << doubleVar << endl;

    return 0;
}
```

Output

```
Integer Variable: 10
Float Variable: 3.14
Character Variable: A
Boolean Variable: 1
Double Variable: 3.14159
```

Summary

This simple program demonstrates how to create variables, initialize them, and print their values to the console in C++.

Exercise 1.3: Create a variable to store float data type number and integer data type number and print the output.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs (Name of the variables should show their purposes)

Experiment 1.4: Write a C++ program to perform Arithmetic Operations (Addition, Subtraction, Multiplication and Division) accepting 2 inputs from the user.

Algorithm Steps:

1. Start
2. Input:
 - Prompt the user to enter the first number and store it in variable num1.
 - Prompt the user to enter the second number and store it in variable num2.
3. Perform and Display Arithmetic Operations:

- Addition:
 - Calculate the sum of num1 and num2.
 - Display the addition result with an explanatory message.
- Subtraction:
 - Subtract num2 from num1.
 - Display the subtraction result with an explanatory message.
- Multiplication:
 - Multiply num1 by num2.
 - Display the multiplication result with an explanatory message.
- Division:
 - Check if num2 is not equal to zero to avoid division by zero.
 - If true, divide num1 by num2 and display the division result with a message.
 - If false, display an error message indicating "Cannot divide by zero!"

4. End

Code Snippet

cpp

 Copy code

```
#include <iostream>
using namespace std;

int main() {
    // Declare variables to store numbers and the result
    double num1, num2, result;

    // Input: Get two numbers from the user
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;
```

```
// Addition
result = num1 + num2;
cout << "The sum of " << num1 << " and "
    << num2 << " is: " << result << endl;

// Subtraction
result = num1 - num2;
cout << "The difference between " << num1 << " and "
    << num2 << " is: " << result << endl;

// Multiplication
result = num1 * num2;
cout << "The product of " << num1 << " and "
    << num2 << " is: " << result << endl;
```

```
// Division (with a check to prevent division by zero)
if(num2 != 0) {
    result = num1 / num2;
    cout << "The division of " << num1 << " by "
        << num2 << " is: " << result << endl;
} else {
    cout << "Cannot divide "
        << "by zero!" << endl;
}
```

Output: This will prompt the user to input two numbers. Eg. 10, 5

```
Enter first number: 10
Enter second number: 5
The sum of 10 and 5 is: 15
The difference between 10 and 5 is: 5
The product of 10 and 5 is: 50
The division of 10 by 5 is: 2
```

Exercise 1.4: Write a C++ program to perform Arithmetic Operations (Addition, Subtraction, Multiplication and Division) accepting 5 inputs from the user

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

LAB 2: Control Structures and Functions

Objectives:

- Understand and implement decision-making structures like if, if-else, and loop(while,for).
- Understand the distinction between declaring a function prototype and defining a function.
- Grasp how to return values from functions and the role of the return statement.

Experiment 2.1: Conditional Statements (if, if else)

- Using if statement only, write a c++ program to check if a user is eligible to vote.

Algorithm:

1. **Start.**
2. **Include Libraries:**
 - Include the necessary libraries (iostream) for input and output operations.
3. **Setup Namespace:**
 - Use the standard namespace (std) to avoid prefixing every input-output operation with std::.

Begin Main Execution:

4. **Declare Variables:**
 - Declare an integer variable age to store the user's age.
5. **User Input:**

- Prompt the user with the message "Enter your age: ".
- Read the entered age and store it in the age variable.

6. **if Statement Check:**

- Check if the value of age is greater than or equal to 18.

7. **If the condition is true:**

- Print the message "You are eligible to vote." to the console.

8. **End Note:**

- Print the message "Program ended." to indicate the end of the program's execution.

End Main Execution:

9. Return 0 to signal successful program termination.

10. End.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int age;

    // Prompt the user for input
    cout << "Enter your age: ";
    cin >> age;

    // Use the if statement to check if the user is 18 or older
    if (age >= 18) {
        cout << "You are eligible to vote." << endl;
    }

    cout << "Program ended." << endl;

    return 0;
}
```

Output:

```
Enter your age: 20
You are eligible to vote.
Program ended.
```

Summary:

- The program prompts the user to enter their age.
 - Using the if statement, it checks if the age entered is 18 or older.
 - If the condition is met (age is 18 or older), the program will print out that the user is eligible to vote.
 - Regardless of the condition, the program will then print "Program ended."
 - When executed, depending on the age input provided by the user, the program will either state the user's eligibility to vote or just end the program.
- a. **Using the if else statement, write a program to check if a user is eligible to vote and provide a response if eligible or not.**

Algorithm:

1. Start.
2. **Include Necessary Libraries:**
 - Include the iostream library for input and output operations.
3. **Setup Namespace:**
 - Use the standard namespace (std) to avoid prefixing input-output operations with std::.

Begin Main Execution:

4. **Variable Declaration:**

- Declare an integer variable named age to store the user's age.

5. **User Input:**

- Display the message "Enter your age: " on the console.
- Accept the user's input for the age variable.

6. **Teenager Check:**

- Check if age is greater than or equal to 13 and less than or equal to 19:
 - a. **If both conditions are true:**
 - Display the message "Person is Teenager".
 - b. **Otherwise:**
 - Display the message "Person is not a Teenager".

7. **Voting Eligibility Check:**

- Check if age is greater than or equal to 18:
 - a. **If true:**
 - Display the message "Person is eligible for voting".
 - b. **Otherwise:**
 - Display the message "Person is not eligible for voting".

End Main Execution:

8. Return 0 to signal successful program termination.
9. End.

Code:

```

1  #include<iostream>
2  using namespace std;
3
4  int main()
5  {
6      int age;
7
8      cout<<"Enter your age: ";
9      cin>>age;
10
11     //person is teenager or not
12     //>=13 and <=19
13     if(age>=13 && age<=19)
14     {
15         cout<<"Person is Teenager"<<endl;
16     }
17     else
18     {
19         cout<<"Person is not a Teenager"<<endl;
20     }
21

```

```

21
22     //condition to check voting eligibility
23     if(age>=18)
24     {
25         cout<<"Person is eligible for voting"<<endl;
26     }
27     else
28     {
29         cout<<"Person is not eligible for voting"<<endl;
30     }
31
32     return 0;
33 }
34

```

Output:

```

C:\Users\Owner\CLionProjects\C program\Cpp program.exe
Enter your age:16
Person is Teenager
Person is not eligible for voating
Process finished with exit code 0

```

Summary:

Depending on the user's input, the program will display whether the person is a teenager and whether they are eligible to vote.

Exercise 2.1: Write a program that determines whether a given number is even or odd

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Experiment 2.2: Loops (while, for)

- Write a simple C++ program to print number 1 - 5 using for loop

Algorithm:

1. **Start.**
2. **Include Libraries:**
 - Include the necessary libraries (iostream) for input and output operations.
3. **Setup Namespace:**
 - Use the standard namespace (std) to avoid prefixing every input-output operation with std::.

Begin Main Execution:

4. **Output:**

- Print the message "Numbers from 1 to 5:" to the console.

5. For Loop:

- Initialize an integer variable i to 1.

6. While i is less than or equal to 5, repeat the following steps:

- a. Print the value of i followed by a space.
- b. Increment the value of i by 1.

7. Formatting:

- Print a newline character to separate the output.

End Main Execution:

8. Return 0 to signal successful program termination.
9. End.

Code:

```
cpp

#include <iostream>
using namespace std;

int main() {
    // Print numbers from 1 to 5 using a for loop
    cout << "Numbers from 1 to 5:" << endl;

    for(int i = 1; i <= 5; i++) {
        cout << i << " ";
    }

    cout << endl; // Print a newline for formatting

    return 0;
}
```

Output:

```
Numbers from 1 to 5:
1 2 3 4 5
```

Summary:

- The program includes the `iostream` header to enable input-output operations using `cout`.
- Inside the main function, we have a for loop that initializes the integer `i` to 1. The loop will continue iterating as long as `i` is less than or equal to 5. After each iteration, `i` is incremented by 1.
- Within the loop, the program prints the current value of `i`.
- After the loop concludes, a newline is printed for formatting.
- When executed, the program will display the numbers 1 to 5 on the console.

a. Write a C++ program to find the sum of positive numbers.

Algorithm:

1. **Start.**
2. **Include Necessary Libraries:**
 - Include the `iostream` library for input and output operations.
3. **Setup Namespace:**
 - Use the standard namespace (`std`) to simplify input-output operations.

Begin Main Execution:

4. **Variable Initialization:**
 - Declare an integer variable named `number` to store the user's input.

- Initialize an integer variable named sum to 0. This will store the sum of the positive numbers.

5. User Input:

- Display the message "Enter a number: " on the console.
- Accept the user's input for the number variable.

6. Positive Number Check:

- While the value of number is greater than or equal to 0, execute the following steps:
 - a. Addition:
 - Add the value of `number` to `sum`.
 - b. Request Further Input:
 - Display the message "Enter a number: " again to prompt the user for another input.
 - Accept the next input for the number variable.

7. Exit Loop and Display Sum:

- Once the user enters a negative number (or number becomes negative), exit the loop.
- Display the message "\nThe sum is " followed by the value of sum to show the total sum of the positive numbers entered.

End Main Execution:

8. Return 0 to indicate successful program termination.
9. End.

Code:

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int number;
5      int sum = 0;
6
7      // take input from the user
8      cout << "Enter a number: ";
9      cin >> number;
10
11     while (number >= 0) {
12         // add all positive numbers
13         sum += number;
14         // take input again if the number is positive
15         cout << "Enter a number: ";
16         cin >> number;
17     }
18
19     // display the sum
20     cout << "\nThe sum is " << sum << endl;
21
22     return 0;
23 }
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
Enter a number:2
Enter a number:4
Enter a number:1
Enter a number:0
Enter a number:5
Enter a number:-5

The sum is 12

Process finished with exit code 0
```

Summary:

In this scenario, the user enters the numbers 2, 4, 1, 0 and 5 sequentially, all of which are positive and thus are added to the sum. When the user enters the negative number -5, the program exits the loop and displays the sum of the positive numbers, which is 12. The negative number -5 is not included in the sum.

Exercise 2.2: Write a program using a loop to calculate the sum of first n natural numbers.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Experiment 2.3: Functions, Parameters, and Return Values

- Write a simple C++ program to declare a function and call the function.

Algorithm:

- **Start.**
- **Include Necessary Libraries:**
 - Include the iostream library for input and output operations.
- **Setup Namespace:**
 - Use the standard namespace (std) to simplify input-output operations.
- **Function Declaration:**

- Function Name: greet
- **Function Task:**
 - Display the message "Hello. This is my first C++ function!" on the console.

Begin Main Execution:

- Function Call:
 - Call the greet function to execute its task.

End Main Execution:

- Return 0 to indicate successful program termination.
- End.

Code:

```
1  #include <iostream>
2  using namespace std;
3
4  // declaring a function
5  void greet() {
6      cout << "Hello. This is my first C++ Function!";
7  }
8
9  int main() {
10
11      // calling the function
12      greet();
13
14      return 0;
15  }
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
Hello. This is my first C++ Function!
Process finished with exit code 0
```

Summary:

The program simply calls the greet function which prints "Hello. This is my first C++ !" to the console.

- **Write a simple C++ program to display number using function with parameters**

Algorithm:

1. Start.

2. Include Necessary Libraries:

- Include the iostream library for input and output operations.

3. Setup Namespace:

- Use the standard namespace (std) to simplify input-output operations.
- Function Declaration:
 - a. Function Name: displayNum
- Parameters: Two parameters, n1 of integer type and n2 of float type.

4. Function Task:

- Display the message "The int number is " followed by the value of n1.
- Display the message "The double number is " followed by the value of n2.

Begin Main Execution:

5. Variable Initialization:

- Initialize an integer variable named num1 with the value 5.
- Initialize a double variable named num2 with the value 5.5.

6. Function Call:

- Call the displayNum function, passing num1 and num2 as arguments.

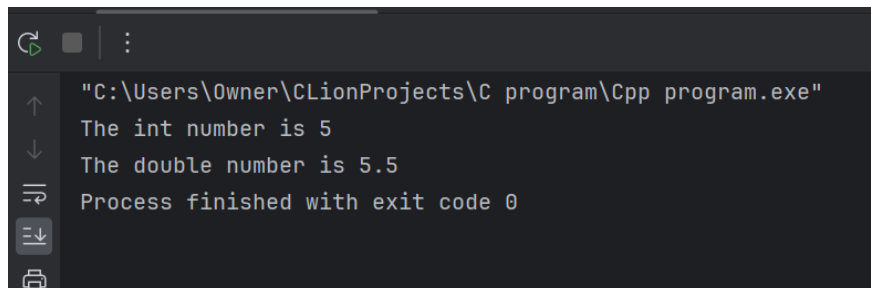
End Main Execution:

7. Return 0 to indicate successful program termination.
8. End.

Code:

```
2
3  #include <iostream>
4  using namespace std;
5
6  // display a number
7  void displayNum(int n1, float n2) {
8      cout << "The int number is " << n1 << endl;
9      cout << "The double number is " << n2;
10 }
11
12 int main() {
13
14     int num1 = 5;
15     double num2 = 5.5;
16
17     // calling the function
18     displayNum( n1: num1, n2: num2);
19
20     return 0;
21 }
```

Output:



```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
The int number is 5
The double number is 5.5
Process finished with exit code 0
```

Summary:

The program displays the values of an integer and a double variable on the same line, resulting in the output: "The int number is 5" and "The double number is 5.5".

- **Write a simple C++ program to add two numbers using a function**

Algorithm:

1. **Start.**
2. **Include Necessary Libraries:**
 - Include the iostream library for input and output operations.
3. **Setup Namespace:**
 - Use the standard namespace std for simplicity.
4. **Function Prototype:**
 - Declare the prototype for the function sum that takes two integers as parameters and returns an integer.

Begin Main Execution:

5. **Variable Declaration:**
 - Declare three integer variables: x, y, and result.
6. **User Input:**
 - Display the prompt "Enter two numbers: ".
 - Accept user input for the variables x and y.
7. **Function Call:**
 - Call the sum function with x and y as arguments.
 - Store the returned value in the result variable.
8. **Display Result:**

- Print "Sum of numbers is: " followed by the value of result.

End Main Execution:

9. Return 0 for successful program termination.

10. Function Definition:

- Define the sum function to:
 - Take two integer parameters a and b.
 - Calculate the sum of a and b, storing it in the integer variable z.
 - Return the value of z.

11. End.

Code:

```
1  #include <iostream>
2  using namespace std;
3      //Function Prototype
4  int sum(int a, int b);
5
6      //main function
7  int main()
8  {
9      int x, y;
10     int result;
11     cout << "Enter two numbers: ";
12     cin >> x >> y;
13     result = sum(a: x, b: y);
14     cout << "Sum of numbers is: " << result << "\n";
15     return 0;
16 }
17     //Function Definition
18 int sum(int a, int b)
19 {
20     int z;
21     z = a + b;
22     return z;
23 }
```

Output: example user enter 12, 8

```
Enter two numbers: 12 8  
Sum of numbers is: 20
```

Summary:

In this scenario, the user enters 12 and 8 as the two numbers. The program then calculates and displays their sum, which is 20.

Exercise 2.3: Implement a guessing game where the user has to guess a random number within a specified range

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

LAB 3: Arrays and Strings

Objectives:

- Understand how to initialize arrays during declaration.
- Learn how to work with one-dimensional arrays and multidimensional arrays.
- Understand how to access and modify individual characters in a string.

Experiment 3.1 Working with One-Dimensional Arrays

- Write a simple C++ program to print the elements of the array

Algorithm:

1. **Start.**
2. **Include Necessary Libraries:**
 - Include the iostream library for input and output operations.
3. **Setup Namespace:**
 - Use the standard namespace std for simplifying code.

Begin Main Execution:

4. **Array Declaration and Initialization:**
 - Declare a one-dimensional integer array named numbers with a size of 5.
 - Initialize the array with the values {10, 20, 30, 40, 50}.
5. **Print the Array Elements:**
 - Display a message "Elements of the array:".
 - Initialize a loop variable i to 0.
 - Start a loop (for loop) that iterates until i is less than 5:
 - a. Within the loop, print the statement "Element at index i: " followed by the value of the array at index i.
 - b. Increment i by 1.

End Main Execution:

6. Return 0 to signify successful termination of the program.
7. End.

Code:


```
#include <iostream>
using namespace std;

int main() {
    // Declare and initialize a one-dimensional array
    int numbers[5] = {10, 20, 30, 40, 50};

    // Print the elements of the array
    cout << "Elements of the array:" << endl;
    for (int i = 0; i < 5; i++) {
        cout << "Element at index " << i << ": " << numbers[i] << endl;
    }

    return 0;
}
```

Output:

```
Elements of the array:
Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
Element at index 3: 40
Element at index 4: 50
```

Summary:

In this program, we've declared a one-dimensional array named `numbers` of size 5 and initialized it with some values. We then use a `for` loop to iterate over the array and print its elements.

Exercise 3.1: : Write a program to find the sum and average of elements in an array.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.

- Use descriptive variables in your programs

Experiment 3.2: Multidimensional Arrays

- Write a C++ program to print elements of 2x3 two-dimensional arrays.

Algorithm:

1. **Start.**
2. **Include Necessary Libraries:**
 - Include the iostream library for input and output operations.
3. **Setup Namespace:**
 - Use the standard namespace std for simplifying code.

Begin Main Execution:

4. **Array Declaration and Initialization:**
 - Declare a two-dimensional integer array named numbers with dimensions 2x3.
 - Initialize the array with values: {1, 2, 3}, {4, 5, 6}
5. **Print the Array Elements:**
 - Display the message "Elements of the 2D array:".
 - Initialize an outer loop variable i to 0.
 - Start an outer loop (for loop) that iterates until i is less than 2:
 - a. Initialize an inner loop variable j to 0.
 - b. Start an inner loop (for loop) that iterates until j is less than
 - Within the inner loop, print the statement "Element at position [i][j]: " followed by the value of the array at indices i and j.

- Increment j by 1.
- c. Increment i by 1.

End Main Execution:

6. Return 0 to signify successful termination of the program.
7. End.

Code:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      // Declare and initialize a 2x3 two-dimensional array
6      int numbers[2][3] = {
7          { [0]: 1, [1]: 2, [2]: 3},
8          { [0]: 4, [1]: 5, [2]: 6}
9      };
10
11     // Print the elements of the 2D array
12     cout << "Elements of the 2D array:" << endl;
13     for (int i = 0; i < 2; i++) {
14         for (int j = 0; j < 3; j++) {
15             cout << "Element at position [" << i << "][" << j << "]: " << numbers[i][j] << endl;
16         }
17     }
18
19     return 0;
20 }
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
Elements of the 2D array:
Element at position [0][0]: 1
Element at position [0][1]: 2
Element at position [0][2]: 3
Element at position [1][0]: 4
Element at position [1][1]: 5
Element at position [1][2]: 6
```

```
Process finished with exit code 0
```

In this program:

- We've declared a two-dimensional array named numbers of size 2x3 and initialized it with some values.
- We then use nested for loops to iterate over the array rows and columns to print its elements.

Exercise 3.2: Implement a program that finds the largest and smallest values in an array.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Experiment 3.3: String Manipulation and C-Style Strings.

- Write a C++ program to read and display an entire line entered by the user.

Algorithm:

1. **Start.**
2. **Include Necessary Libraries:**
 - Include the iostream library for input-output operations.
3. **Setup Namespace:**
 - Use the standard namespace std for simplifying code.

Begin Main Execution:

4. **Variable Declaration:**

- Declare a character array string of size 100 to store the user's input.

5. Prompt User for Input:

- Display a message prompting the user to "Enter a string: ".

6. Read Input:

- Use the `cin.get()` function to read a string from the user. The `cin.get(str, 100)` function reads up to 99 characters from the input and stores it in the `str` array, leaving one space for the null terminator.

7. Display User's Input:

- Display the message "You entered: " followed by the content of `str`.

End Main Execution:

8. Return 0 to signify successful termination of the program.
9. End.

Code:

```
#include <iostream>
using namespace std;

int main()
{
    char str[100];
    cout << "Enter a string: ";
    cin.get(str, 100);

    cout << "You entered: " << str << endl;
    return 0;
}
```

Output: example if the user enters 'Programming is fun.'

```
Enter a string: Programming is fun.  
You entered: Programming is fun.
```

In this program:

- When you run the program, it will prompt you to enter a line of text.
- After you input the text and press Enter, the program will display the text you entered.
- The `getline()` function is used specifically to read a full line of input, which can include spaces.

Exercise 3.3: Implement a program that finds the largest and smallest values in an array.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

LAB 4: Object-Oriented Programming Basics

Objectives:

- Learn how to declare classes and create objects.
- Understand the role of constructors in object initialization
- Learn the significance of destructors in resource deallocation.

Experiment 4.1: Introduction to Classes and Objects

- Write a simple C++ program to demonstrate use of class and object.

Algorithm:

1. Start.
2. **Include Libraries:**
 - a. Include the iostream library to enable input-output operations in C++.
3. **Setup Namespace:**
 - a. Use the standard namespace std to avoid writing std:: before every standard function.
4. **Declare the Dog Class:**
 - a. Declare a public data member name of type string to store the name of the dog.
 - b. Declare a public member function bark():
 - When this function is called, it displays the message indicating the dog's name followed by "says: Woof! Woof!".

Begin Main Execution:

5. Create an object myDog of the Dog class.

6. Assign the string "Buddy" to the name data member of the myDog object.
7. Call the bark() member function of the myDog object to make the dog bark. This will display the message indicating the dog's name followed by its bark.

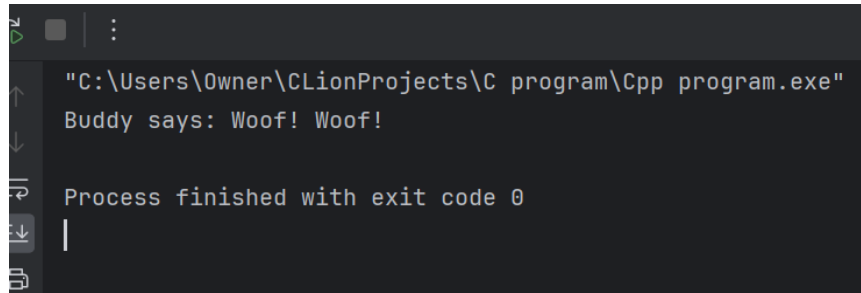
End Main Execution:

8. Return 0 to indicate successful termination of the program.
9. End.

Code:

```
1  #include <iostream>
2  using namespace std;
3  // Declaration of the 'Dog' class
4  class Dog {
5  public:
6      // Data member for the dog's name
7      string name;
8      // Member function for the dog to bark
9      void bark() {
10         cout << name << " says: Woof! Woof!" << endl;
11     }
12 };
13 int main() {
14     // Create a Dog object
15     Dog myDog;
16
17     // Name the dog
18     myDog.name = "Buddy";
19     // Make the dog bark
20     myDog.bark();
21
22     return 0;
23 }
```


Output:



```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
Buddy says: Woof! Woof!

Process finished with exit code 0
```

In this program:

- We've declared a Dog class with a name as its data member and a function bark().
- In the main() function, we create a Dog object named myDog.
- We assign the name "Buddy" to myDog and then make myDog bark by calling its bark() function.

Exercise 4.1: Create a class representing a student with attributes like name, roll number, and grade. Instantiate and display student objects.

Create a class representing a student with attributes like name, roll number, and grade. Instantiate and display student objects.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Experiment 4.2: Constructors and Destructors

- Write a simple C++ program to demonstrate Constructors and Destructors

Algorithm:

1. Start.
2. **Include Libraries:**
 - a. Include the iostream library for standard input-output operations.
3. **Setup Namespace:**
 - a. Use the std namespace to simplify the use of standard C++ functions and classes.
4. **Declare the Car Class:**
 - a. **Private Data Members:**
 - Declare a private string data member named brand to store the brand of the car.
 - b. **Constructor (Car(string b)):**
 - Assign the passed string b to the brand data member.
 - Display a message indicating the car's brand followed by "car created!".
 - c. **Destructor (~Car()):**
 - Display a message indicating the car's brand followed by "car destroyed!".
 - d. **Member Function drive():**
 - Display a message indicating "Driving the" followed by the car's brand and "car".

Begin Main Execution:

5. Create a Car object named myCar with the brand "Toyota".
 - This will invoke the constructor and display the creation message.
6. Call the drive() member function of the myCar object.
 - This will display the driving message.
7. As the main() function reaches its end, the myCar object will go out of scope, automatically invoking the destructor and displaying the destruction message.

End Main Execution:

8. Return 0 to indicate successful termination of the program.
9. End.

Code:

```
#include <iostream>
using namespace std;

class Car {
    private:
        string brand;

    public:
        // Constructor
        Car(string b) {
            brand = b;
            cout << brand << " car created!" << endl;
        }
}
```

```
// Destructor
~Car() {
    cout << brand << " car destroyed!" << endl;
}

void drive() {
    cout << "Driving the " << brand << " car." << endl;
}

};

int main() {
    Car myCar("Toyota");
    myCar.drive();

    // At the end of main(), the destructor will be called automatically.
    return 0;
}
```

Output:

```
Toyota car created!
Driving the Toyota car.
Toyota car destroyed!
```

In this program:

- When the Car object myCar is created with the brand "Toyota", the constructor is called, which prints "Toyota car created!".
- We then call the drive() method to simulate driving the car.
- As the main() function ends and the program scope finishes, the destructor for the myCar object is automatically called, which prints "Toyota car destroyed!".

Exercise 4.2: Extend the student class to calculate and display the grade based on the marks obtained.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Experiment 4.3: Member Functions and Data Members

- Write a C++ program that will demonstrate how to declare, define and access data members and member functions in a class.

Algorithm:

1. Start.
2. **Include Necessary Libraries:**
 - a. Include the iostream for input-output operations.
 - b. Include the string.h library for string operations.
3. **Setup Namespace:**
 - a. Use the std namespace to simplify the usage of standard C++ functions and classes.
4. **Macro Definition:**
 - a. Define a macro MAX_CHAR with a value of 30.
5. **Declare the person Class:**
 - a. **Private Data Members:**

- Declare a private character array named name with a size of MAX_CHAR to store the person's name.
- Declare a private integer named age to store the age of the person.

b. Public Member Functions:

- Function get(char n[], int a):
 - Copy the character array passed as n into the name data member using the 'strcpy' function.
 - Assign the integer a to the age data member.
- Function put():
 - Display "Name: " followed by the name data member.
 - Display "Age: " followed by the age data member.

Begin Main Execution:

6. Create an object of the person class named PER.
7. Call the get() member function of the PER object, passing the string "Manju Tomar" and the integer 23 as arguments.
 - This will set the name and age of the PER object.
8. Call the put() member function of the PER object.
 - This will display the name and age of the PER object.
9. End the main function by returning 0.
10. End.

Code:

```
1  #include <iostream>
2  #include <string.h>
3  using namespace std;
4
5  #define MAX_CHAR 30
6
7  //class definition
8  class person
9  {
10     //private data members
11 private:
12     char name [MAX_CHAR];
13     int age;
14
15     //public member functions
16 public:
17     //function to get name and age
18     void get(char n[], int a)
19     {
20         strcpy( Dest: name , Source: n);
21         age = a;
22     }
23 }
```

```
23
24     //function to print name and age
25     void put()
26     {
27         cout<< "Name: " << name <<endl;
28         cout<< "Age: " <<age <<endl;
29     }
30 };
31
32 //main function
33 int main()
34 {
35     //creating an object of person class
36     person PER;
37
38     //calling member functions
39     PER.get( n: "Manju Tomar", a: 23);
40     PER.put();
41
42     return 0;
43 }
```

Output:

```
Name: Manju Tomar  
Age: 23
```

Exercise 4.3: Design a class for a simple bank account with constructors to initialize account details.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

LAB 5: Advanced OOP Concepts

Objectives:

- Grasp the concept of inheritance, which allows the creation of a new class from an existing class.
- Learn about the ability of a function or method to behave in multiple ways.
- Learn about access specifiers: private, protected, and public.

Experiment 5.1: Inheritance and Polymorphism

- Write a simple C++ example of inheritance where a Vehicle class (base class) will be inherited by a Car class (derived class).

Algorithm:

1. Start.
2. **Include Libraries:**
 - a. Include the iostream library for standard input-output operations.
3. **Setup Namespace:**
 - a. Use the std namespace to simplify the use of standard C++ functions and classes.
4. **Declare the Vehicle Class (Base Class):**
 - a. **Protected Data Members:**
 - Declare a protected string data member named brand to store the brand of the vehicle.
 - b. **Public Member Functions:**
 - **Function setBrand(string b):**

- Assign the passed string b to the brand data member.

- **Function getBrand():**

- Return the value stored in the brand data member.

5. Declare the Car Class (Derived Class):

a. Private Data Members:

- Declare a private integer data member named numberOfDoors to store the number of doors of the car.

b. Public Member Functions:

- **Function setNumberOfDoors(int doors):**

- Assign the passed integer doors to the numberOfDoors data member.

- **Function displayInfo():**

- Display the car's brand (inherited member from the Vehicle class) and its number of doors.

Begin Main Execution:

6. Create an object of the Car class named myCar.
7. Set the brand of the myCar object to "Toyota" using the setBrand function.
8. Set the number of doors of the myCar object to 4 using the setNumberOfDoors function.
9. Display the car's information using the displayInfo function of the myCar object.
10. End the main function by returning 0.
11. End.

Code:

```
1      #include <iostream>
2      using namespace std;
3
4      // Base class: Vehicle
5      class Vehicle {
6      protected:
7          string brand;
8
9      public:
10         void setBrand(string b) {
11             brand = b;
12         }
13
14         string getBrand() {
15             return brand;
16         }
17     };
18
19     // Derived class: Car
20     class Car : public Vehicle {
21     private:
22         int numberOfDoors;
23
24     public:
25         void setNumberOfDoors(int doors) {
26             numberOfDoors = doors;
27         }
28
29         void displayInfo() {
30             cout << "Car Brand: " << brand << endl;
31             cout << "Number of Doors: " << numberOfDoors << endl;
32         }
33     };
34
```

```
34
35 ▶ ~ int main() {
36     Car myCar;
37
38     myCar.setBrand(b: "Toyota");
39     myCar.setNumberOfDoors(doors: 4);
40
41     myCar.displayInfo();
42
43     return 0;
44 }
```

Output:

```
Car Brand: Toyota
Number of Doors: 4
```

In this program:

- The `Vehicle` class is a basic class with a protected member brand and functions to set and get its value.
- The `Car` class is a derived class that inherits the properties of the Vehicle class. It has its own private member `numberOfDoors` and a function `displayInfo()` to showcase the properties of a car.

In this example, you can see that the derived class `Car` has access to the `brand` property of the base class `Vehicle` and also its own specific properties and behaviors

- **Write a simple C++ program to demonstrate polymorphism using a base class Shape and two derived classes `Rectangle` and `Circle`.**

Algorithm:

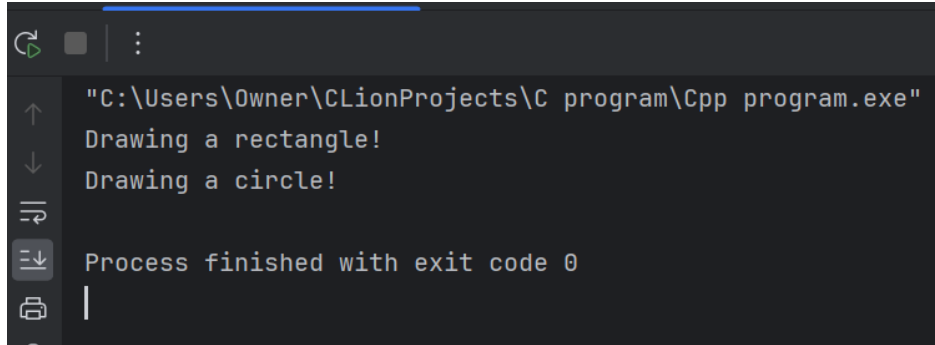
Algorithm Demonstrate Polymorphism:

1. START
2. Create an abstract class named "Shape" with a pure virtual function "draw".
3. Create a derived class named "Rectangle" from the base class "Shape":
 - Override the "draw" function.
 - Inside the "draw" function, display "Drawing a rectangle!".
4. Create another derived class named "Circle" from the base class "Shape":
 - Override the "draw" function.
 - Inside the "draw" function, display "Drawing a circle!".
5. Create a function "DrawShape" that:
 - Accepts a pointer to an object of the base class "Shape".
 - Calls the "draw" function of the passed object.
6. MAIN:
 - Create an object of the "Rectangle" class named "rect".
 - Create an object of the "Circle" class named "circ".
 - Call the "DrawShape" function, passing the "rect" object pointer.
 - Call the "DrawShape" function, passing the "circ" object pointer.
7. END

Code:

```
1  #include <iostream>
2  using namespace std;
3
4  // Base class
5  class Shape {
6  public:
7      virtual void draw() = 0; // Pure virtual function
8  };
9
10 // Derived class 1
11 class Rectangle : public Shape {
12 public:
13     void draw() {
14         cout << "Drawing a rectangle!" << endl;
15     }
16 };
17
18 // Derived class 2
19 class Circle : public Shape {
20 public:
21     void draw() {
22         cout << "Drawing a circle!" << endl;
23     }
24 };
25
26 void DrawShape(Shape* shape) {
27     shape->draw();
28 }
29
30 int main() {
31     Rectangle rect;
32     Circle circ;
33
34     // Using polymorphism
35     DrawShape( shape: &rect);
36     DrawShape( shape: &circ);
37
38     return 0;
39 }
```

Output:



```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"  
Drawing a rectangle!  
Drawing a circle!  
Process finished with exit code 0
```

In the program:

- Shape is an abstract class with a pure virtual function draw().
- Rectangle and Circle are derived classes that override the draw() function to provide their own implementations.
- The DrawShape function takes a pointer to a Shape object and calls its draw() function. This is where the polymorphism takes place: even though the function takes a Shape*, we can pass in any object that derives from Shape and the appropriate draw() function will be called.

Exercise 5.1: Create a base class "Shape" and derive classes for different shapes (e.g., circle, rectangle). Calculate and display their areas using polymorphism.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Experiment 5.2: Encapsulation and Abstraction

- Write a simple C++ program to demonstrate encapsulation using a class `Person`

Algorithm:

1. Start
2. Define a class "Person":
 - a. **Private:**
 - Declare a string variable "name".
 - Declare an integer variable "age".
 - b. **Public:**
 - i. Define a constructor that accepts a string "n" and an integer "a":
 - Initialize "name" with "n".
 - Initialize "age" with "a".
 - ii. Define a getter function "getName" that returns the value of "name".
 - iii. Define a setter function "setName" that accepts a string "n":
 - Assign "n" to "name".
 - iv. Define a getter function "getAge" that returns the value of "age".
 - v. Define a setter function "setAge" that accepts an integer "a":
 - Check if "a" is non-negative.
 - If true, assign "a" to "age".
3. **Main:**
 - Create an object of "Person" class with initial values "Alice" for name and 25 for age, and name it "person".
 - Display the name and age of "person" using the getter functions.

- Change the name of "person" to "Bob" using the setter function.
- Change the age of "person" to 30 using the setter function.
- Display the updated name and age of "person" using the getter functions.

4. End

Code:

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // Class definition
6  class Person {
7  private:
8      string name;
9      int age;
10
11 public:
12     // Constructor
13     Person(string n, int a) : name(n), age(a) {}
14
15     // Getter for name
16     string getName() const {
17         return name;
18     }
19
20     // Setter for name
21     void setName(string n) {
22         name = n;
23     }
24
25     // Getter for age
26     int getAge() const {
27         return age;
28     }
29
30     // Setter for age
31     void setAge(int a) {
32         if (a >= 0) // Simple validation to ensure age is non-negative
33             age = a;
34     }
35 };
36
```

```
36
37 > int main() {
38     Person person( n: "Alice", a: 25);
39
40     cout << person.getName() << " is " << person.getAge() << " years old." << endl;
41
42     person.setName( n: "Bob");
43     person.setAge( a: 30);
44
45     cout << person.getName() << " is " << person.getAge() << " years old." << endl;
46
47     return 0;
48 }
```

Output:

```
Alice is 25 years old.
Bob is 30 years old.
```

In this program:

- We've created a `Person` class with private attributes `name` and `age`.
- Public methods (getName, setName, getAge, and setAge) allow controlled access and modification of these attributes.
- The setters provide a level of validation (like ensuring the age is non-negative).

a. Write a simple program using a base class `Vehicle` to demonstrate abstraction

Algorithm:

1. START

2. Define an abstract class "Vehicle":

a. Private:

i. Declare an integer variable "speed".

b. Public:

- i. Define a constructor that accepts an integer "s":
 - Initialize "speed" with "s".
- ii. Define a function "setSpeed" that accepts an integer "s":
 - Assign "s" to "speed".
- iii. Define a getter function "getSpeed" that returns the value of "speed".
- iv. Define a pure virtual function "displayType".

3. Define a class "Car" derived from the abstract class "Vehicle":

a. Public:

- i. Define a constructor that accepts an integer "s" and initializes the base class with "s".
- ii. Override the "displayType" function:
 - Display "This is a car running at [speed value] km/h."

4. Define a class "Boat" derived from the abstract class "Vehicle":

a. Public:

- i. Define a constructor that accepts an integer "s" and initializes the base class with "s".
- ii. Override the "displayType" function:
 1. Display "This is a boat running at [speed value] knots."

5. Main:

- a. Create an object of "Car" class with initial value 80 for speed, and name it "myCar".
- b. Create an object of "Boat" class with initial value 20 for speed, and name it "myBoat".

- c. Call the "displayType" function of the "myCar" object.
- d. Call the "displayType" function of the "myBoat" object.

6. End

Code:

```

1  #include <iostream>
2  using namespace std;
3
4  // Abstract base class
5  class Vehicle {
6  private:
7      int speed;
8
9  public:
10     // Constructor
11     Vehicle(int s) : speed(s) {}
12
13     // Set speed
14     void setSpeed(int s) {
15         speed = s;
16     }
17
18     // Get speed
19     int getSpeed() const {
20         return speed;
21     }

```

```

21     }
22
23     // Pure virtual function to provide interface for derived classes
24     virtual void displayType() = 0;
25 };
26 // Derived class 1
27 class Car : public Vehicle {
28 public:
29     // Constructor
30     Car(int s) : Vehicle(s) {}
31
32     void displayType() {
33         cout << "This is a car running at " << getSpeed() << " km/h." << endl;
34     }
35 };
36 // Derived class 2

```

```
36 // Derived class 2
37 class Boat : public Vehicle {
38 public:
39     // Constructor
40     Boat(int s) : Vehicle(s) {}
41
42 void displayType() {
43     cout << "This is a boat running at " << getSpeed() << " knots." << endl;
44 }
45 };
46
47 int main() {
48     Car myCar( 80);
49     Boat myBoat( 20);
50
51     myCar.displayType();
52     myBoat.displayType();
53
54     return 0;
55 }
```

Output:

```
This is a car running at 80 km/h.
This is a boat running at 20 knots.
```

In this program:

- Vehicle is an abstract class because it contains a pure virtual function `displayType()`. The class also encapsulates the speed attribute, providing a good example of both encapsulation and abstraction.
- Car and Boat are concrete classes derived from 'Vehicle'. They provide specific implementations for the `displayType()` function.
- The user interacts with the objects through the base class interface (i.e., through the abstraction), without needing to know the intricate details of each derived class.

Exercise 5.2: Implement a basic employee hierarchy with a base class and derived classes for various employee types.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

LAB 6: Pointers and Dynamic Memory Allocation

Objectives:

- Learn how to declare and initialize pointers.
- Understand the use of the arrow operator ($\rightarrow$) with pointers to objects.
- Learn to use the new operator for memory allocation in C++.
- Familiarize with the delete operator for freeing memory in C++.

Experiment 6.1: Pointers and References

- Write a simple C++ program that declares a variable, initializes it with a value from the user, creates a pointer to that variable, and then displays the values and addresses of both the variable and the pointer.

Algorithm:

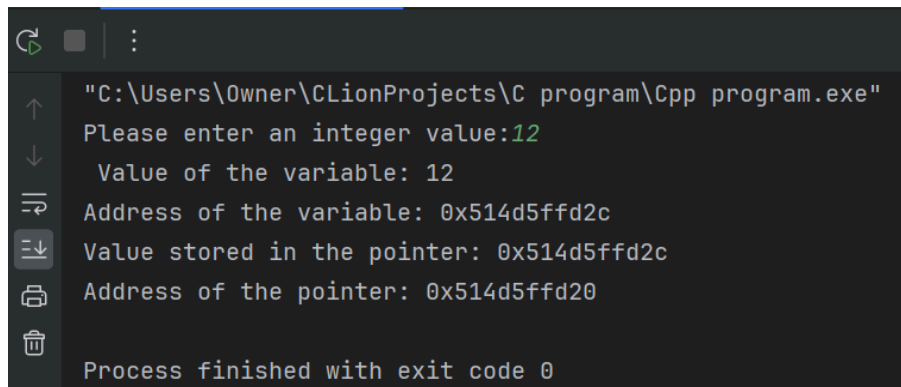
1. START
2. Declare an integer variable named "value".
3. OUTPUT: "Please enter an integer value: "
4. INPUT: value from the user and store it in "value".
5. Declare an integer pointer named "pointerToValue" and initialize it with the address of "value".
6. OUTPUT: The value of "value".
7. OUTPUT: The address of "value".
8. OUTPUT: The value stored in "pointerToValue" (which is the address of "value").
9. OUTPUT: The address of "pointerToValue".
10. END

Code:

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      // Declare an integer variable
5      int value;
6
7      // Accept a value from the user
8      cout << "Please enter an integer value: ";
9      cin >> value;
10
11     // Create a pointer to the variable
12     int* pointerToValue = &value;
13
14     // Display the value and address of the variable
15     cout << "Value of the variable: " << value << endl;
16     cout << "Address of the variable: " << &value << endl;
17
18     // Display the value (which is an address) and address of the pointer
19     cout << "Value stored in the pointer: " << pointerToValue << endl;
20     cout << "Address of the pointer: " << &pointerToValue << endl;
21
22     return 0;
23 }
```

Output: example a user enter 12



```

"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
Please enter an integer value:12
Value of the variable: 12
Address of the variable: 0x514d5ffd2c
Value stored in the pointer: 0x514d5ffd2c
Address of the pointer: 0x514d5ffd20
Process finished with exit code 0
```

In this example:

- The user entered 12 for the variable value.
- The address of the variable value (for this execution) is 0x514d5ffd2c.
- The value stored in the pointer `pointerToValue` is the address of the variable value, which is 0x514d5ffd2c.

- The address where the pointer `pointerToValue` itself is stored is `0x514d5ffd20`.
- a. **Write a simple C++ program that declares a variable, initializes it with a value from the user, creates a reference to that variable, and then displays the values and addresses of both the variable and the reference.**

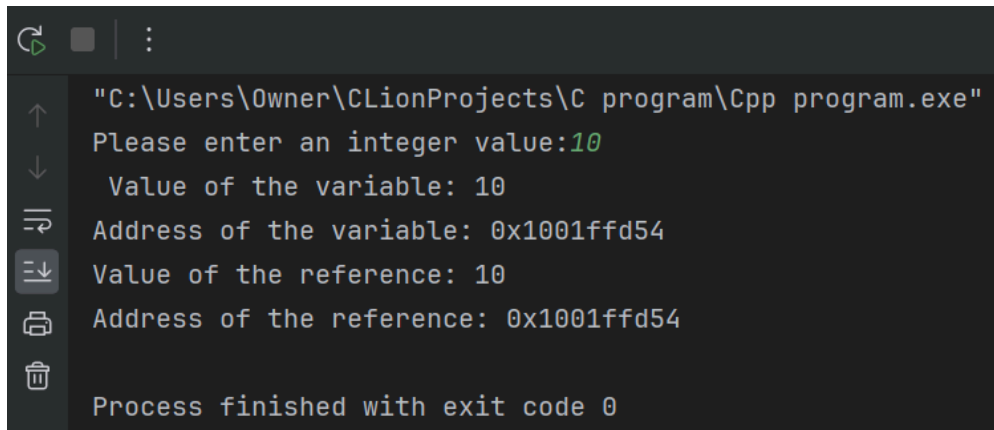
Algorithm:

1. START
2. Declare an integer variable named "value".
3. OUTPUT "Please enter an integer value: "
4. INPUT value
5. Declare a reference to an integer named "referenceToValue" and bind it to "value".
6. OUTPUT the value of "value".
7. OUTPUT the address of "value".
8. OUTPUT the value of "referenceToValue".
9. OUTPUT the address of "referenceToValue".
10. END

Code:

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      // Declare an integer variable
5      int value;
6      // Accept a value from the user
7      cout << "Please enter an integer value: ";
8      cin >> value;
9
10     // Create a reference to the variable
11     int& referenceToValue = value;
12
13     // Display the value and address of the variable
14     cout << "Value of the variable: " << value << endl;
15     cout << "Address of the variable: " << &value << endl;
16
17     // Display the value and address of the reference
18     // Note: The reference should have the same address as the variable it references
19     cout << "Value of the reference: " << referenceToValue << endl;
20     cout << "Address of the reference: " << &referenceToValue << endl;
21
22     return 0;
23 }
```

Output: example a user enter 10



```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
Please enter an integer value:10
Value of the variable: 10
Address of the variable: 0x1001ffd54
Value of the reference: 10
Address of the reference: 0x1001ffd54
Process finished with exit code 0
```

In this example:

- The user entered 10 for the variable value.
- The address of the variable value (for this particular execution) is 0x1001ffd54.

- The value of the reference `referenceToValue` matches the value of the variable, which is 10.
- The address of the reference `referenceToValue` is the same as the address of the variable value since a reference acts as an alias to the variable.

Exercise 6.1: Write a program that dynamically allocates an array of integers of a user-specified size using the 'new' keyword. Populate the array with user input and then display the sum of the elements. Don't forget to use 'delete' to free the allocated memory.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Activity:

Experiment 6.2: Dynamic Memory Allocation (new and delete)

→ Write a simple C++ program that demonstrates the dynamic allocation and deallocation of memory using `new` and `delete`

Algorithm:

1. START
2. Declare a pointer to an integer named "p" and initialize it to NULL.
3. Dynamically request memory space for an integer variable using the "new" operator with "nothrow" option:

- Assign the allocated address to the pointer "p".
 - If allocation fails (pointer "p" remains NULL), OUTPUT "allocation of memory failed\n".
 - Otherwise:
 - Store the integer value 29 in the allocated memory.
 - OUTPUT the value stored in the memory using the pointer "p".
4. Dynamically allocate memory for a floating-point variable and initialize it with the value 75.25:
- Declare a pointer to float named "r".
 - Assign the allocated address to the pointer "r".
 - OUTPUT the value of the float variable using the pointer "r".
5. Declare an integer "n" with the value 5.
6. Dynamically request a block of memory enough for "n" integers using the "new" operator with "nothrow" option:
- a. Declare a pointer to integer named "q".
 - b. Assign the allocated memory block's address to "q".
 - c. If allocation fails (pointer "q" remains NULL), OUTPUT "allocation of memory failed\n".
 - d. Otherwise:
 - Loop from $i = 0$ to $n-1$: Store the integer value $(i + 1)$ in the i th location of the memory block.
 - OUTPUT "Value store in block of memory: ".
 - Loop from $i = 0$ to $n-1$: OUTPUT the integer value at the i th location of the memory block followed by a space.

7. Deallocate the memory allocated for the integer variable by deleting pointer "p".
8. Deallocate the memory allocated for the float variable by deleting pointer "r".
9. Deallocate the memory block allocated for "n" integers by deleting the array pointer "q".
10. END

Code:

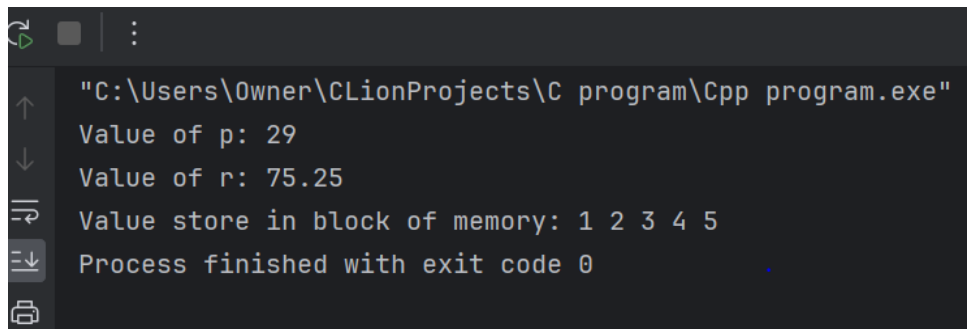
```
1  #include <iostream>
2  using namespace std;
3  int main ()
4  {
5      // Pointer initialization to null
6      int* p = NULL;
7
8      // Request memory for the variable
9      // using new operator
10     p = new(nothrow) int;
11     if (!p)
12         cout << "allocation of memory failed\n";
13     else
14     {
15         // Store value at allocated address
16         *p = 29;
17         cout << "Value of p: " << *p << endl;
18     }
19     // Request block of memory using new operator
20     float *r = new float(75.25);
21
22     cout << "Value of r: " << *r << endl;
```

```

24 // Request block of memory of size n
25 int n = 5;
26 int *q = new(nothrow) int[n];
27
28 if (!q)
29     cout << "allocation of memory failed\n";
30 else
31 {
32     for (int i = 0; i < n; i++)
33         q[i] = i+1;
34
35     cout << "Value store in block of memory: ";
36     for (int i = 0; i < n; i++)
37         cout << q[i] << " ";
38 }
39 // freed the allocated memory
40 delete p;
41 delete r;
42
43 // freed the block of allocated memory
44 delete[] q;
45
46 return 0;
47

```

Output:



```

"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
Value of p: 29
Value of r: 75.25
Value store in block of memory: 1 2 3 4 5
Process finished with exit code 0

```

In this program:

- Memory is allocated for an integer, and it is initialized with the value 29.
So, "Value of p: 29" is printed.
- Memory is allocated for a float, and it is initialized with the value 75.25.
So, "Value of r: 75.25" is printed.

- Memory is allocated for an array of 5 integers. The integers from 1 to 5 are stored in this block of memory. Then, "Value store in block of memory: 1 2 3 4 5" is printed.

Exercise 6.2: Implement a dynamic stack data structure using dynamic memory allocation (new and delete).

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Activity: Implement a dynamic stack data structure using dynamic memory allocation (new and delete). Allow the user to push and pop elements onto and from the stack, and display the contents of the stack after each operation.

Experiment 6.3: Working with Pointers and Objects.

- Create a class representing a "Book" with attributes like title, author, and ISBN. Use dynamic memory allocation to create an array of book objects. Write a program that allows the user to input information for multiple books and then displays the details of all the books.

Algorithm:

1. START

2. Define a class "Book" with private attributes: title, author, and ISBN.

a. Create a default constructor that initializes the attributes with empty values.

b. Create a parameterized constructor for custom initialization.

c. Create a method "input":

i. OUTPUT a prompt for the book title and INPUT the title.

ii. OUTPUT a prompt for the author and INPUT the author.

iii. OUTPUT a prompt for the ISBN and INPUT the ISBN.

d. Create a method "display" to OUTPUT the book's details.

3. In the MAIN function:

a. Declare an integer "n".

b. OUTPUT "How many books do you want to enter? "

c. INPUT n.

d. Dynamically allocate memory for an array of "Book" objects of size "n".

e. For i = 0 to n-1:

i. OUTPUT a prompt for the details of book i.

ii. Call the "input" method for the ith book.

f. OUTPUT the details of all entered books:

i. OUTPUT "Book Details:\n"

ii. For i = 0 to n-1:

1. Call the "display" method for the ith book.

g. Deallocate the dynamically allocated memory for the array of "Book" objects.

4. END

Codes:

```

1  <#include <iostream>
2  <#include <string>
3  using namespace std;
4
5  <class Book {
6  private:
7      string title;
8      string author;
9      string ISBN;
10
11 public:
12     // Default constructor
13     Book() : title(s: ""), author(s: ""), ISBN(s: "") {}
14
15     // Parameterized constructor
16     Book(const string& t, const string& a, const string& i) : title(t), author(a), ISBN(i) {}
17

```

```

17
18     // Function to input book details
19     void input() {
20         cout << "Enter book title: ";
21         cin.ignore(); // Clear any leftover characters in the buffer
22         getline(&: cin, &: title);
23
24         cout << "Enter author: ";
25         getline(&: cin, &: author);
26
27         cout << "Enter ISBN: ";
28         cin >> ISBN;
29     }
30
31     // Function to display book details
32     void display() const {
33         cout << "Title: " << title << ", Author: " << author << ", ISBN: " << ISBN << endl;
34     }
35 };

```

```
36
37  int main() {
38      int n;
39
40      cout << "How many books do you want to enter? ";
41      cin >> n;
42
43      // Dynamically allocate memory for n books
44      Book* books = new Book[n];
45
46      // Input details for each book
47      for (int i = 0; i < n; i++) {
48          cout << "\nEnter details for book " << i+1 << ":\n";
49          books[i].input();
50      }
51
```

```
52      // Display details of all books
53      cout << "\nBook Details:\n";
54      for (int i = 0; i < n; i++) {
55          books[i].display();
56      }
57
58      // Deallocate memory
59      delete[] books;
60
61      return 0;
62 }
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
How many books do you want to enter?2

Enter details for book 1:
Enter book title:Americal Nighmare
Enter author:Cody Rhodes
Enter ISBN:7780747532699

Enter details for book 2:
Enter book title:The Bloodline
Enter author:Jey Uso
Enter ISBN:4467546753269

Book Details:
Title: Americal Nighmare, Author: Cody Rhodes, ISBN: 7780747532699
Title: The Bloodline, Author: Jey Uso, ISBN: 4467546753269

Process finished with exit code 0
```

Exercise 6.3: Create a class representing a simple "Student" with attributes

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Activity: Create a class representing a simple "Student" with attributes like name, roll number, and age. Use dynamic memory allocation to create an array of student objects. Write a program that allows the user to input information for multiple students and then displays the details of all students

LAB 7: File Handling and Exception Handling

Objectives:

- Understand reading from and writing to text files.
- Grasp how to read from and write to text files using ``ifstream`` and ``ofstream``.
- Grasp the usage and significance of try, catch, and throw constructs.
- Learn to write basic exception handlers to catch and handle exceptions.

Experiment 7.1: Reading and Writing Files

- Create a program that reads data from a text file called "input.txt" and writes it to another file called "output.txt". Ensure that you handle file open and close operations gracefully.

Algorithm:

1. START
2. Open "input.txt" for reading into an input stream called "inputFile".
3. Open "output.txt" for writing into an output stream called "outputFile".
4. IF "inputFile" failed to open THEN
 - Display an error message: "Error: Unable to open input.txt for reading."
 - EXIT with code 1.
5. IF "outputFile" failed to open THEN
 - Display an error message: "Error: Unable to open output.txt for writing."
 - EXIT with code 1.
6. WHILE there are more lines in "inputFile":
 - Read a line from "inputFile" into a string called "line".

- Write "line" to "outputFile".
7. Close "inputFile".
 8. Close "outputFile".
 9. Display the message: "Data has been transferred from input.txt to output.txt."
 10. END

Code:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream inputFile("input.txt");    // Open the input file for reading
    ofstream outputFile("output.txt"); // Open the output file for writing

    // Check if input file opened successfully
    if (!inputFile.is_open()) {
        cerr << "Error: Unable to open input.txt for reading." << endl;
        return 1;
    }
}
```

```
// Check if output file opened successfully
if (!outputFile.is_open()) {
    cerr << "Error: Unable to open output.txt for writing." << endl;
    return 1;
}

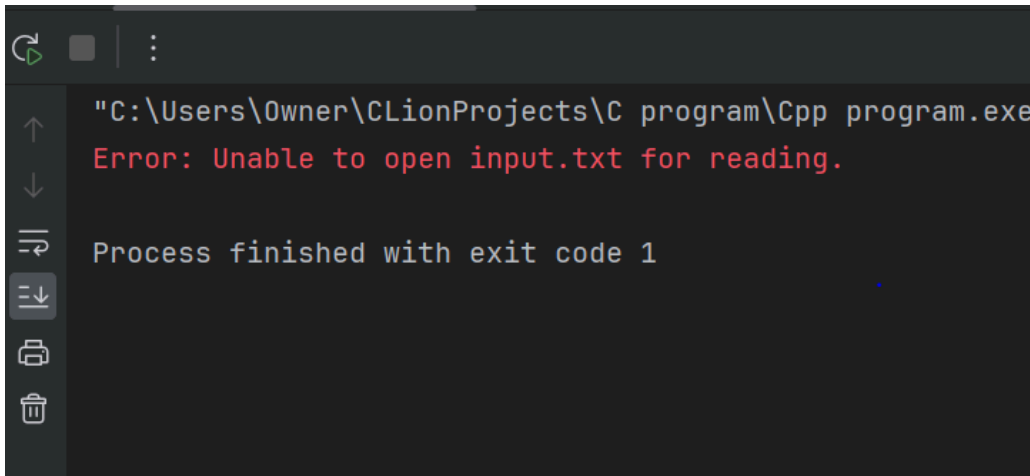
string line;
while (getline(inputFile, line)) { // Read each line from input file
    outputFile << line << endl;    // Write the line to the output file
}

// Close both files
inputFile.close();
outputFile.close();

cout << "Data has been transferred from input.txt to output.txt." << endl;

return 0;
}
```

Output: If there's an issue with opening input.txt, the output will be:



```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe
Error: Unable to open input.txt for reading.

Process finished with exit code 1
```

In this program, it

- Opens "input.txt" for reading and "output.txt" for writing.
- Checks if both files opened successfully. If there's an error in opening any of the files, it outputs an error message.

- Reads each line from "input.txt" and writes it to "output.txt".
- Closes both files after transferring the data.
- Notifies the user that the data transfer is complete.

Exercise 7.1: Write a program that prompts the user to enter two numbers and performs division. Implement exception handling to catch and handle any division by zero exception. Display an error message if such an exception occurs.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Activity: Write a program that prompts the user to enter two numbers and performs division.

Experiment 7.2: Exception Handling (try, catch, throw)

- Write a program that uses exception handling to catch and handle a division by zero exception. Prompt the user to enter two numbers and then perform division. If a division by zero occurs, catch the exception and display an error message.

Algorithm:

1. START
2. Declare three floating-point variables: numerator, denominator, and result.
3. OUTPUT "Enter the numerator: "

4. INPUT numerator
5. OUTPUT "Enter the denominator: "
6. INPUT denominator
7. BEGIN TRY block:
 - a. IF denominator equals 0.0 THEN
 - i. THROW a runtime error with the message "Denominator is zero. Division by zero is not allowed!"
 - b. Compute the division: $\text{result} = \text{numerator} / \text{denominator}$
 - c. OUTPUT the value of result prefixed with "Result: "
8. BEGIN CATCH block for catching a runtime error:
 - a. OUTPUT the caught error message with the prefix "Error: "
9. END

Code:

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      double numerator, denominator, result;
5
6      cout << "Enter the numerator: ";
7      cin >> numerator;
8
9      cout << "Enter the denominator: ";
10     cin >> denominator;
11     try {
12         if (denominator == 0.0)
13             throw runtime_error("Denominator is zero. Division by zero is not allowed!");
14
15         result = numerator / denominator;
16         cout << "Result: " << result << endl;
17     }
18     catch (const runtime_error& e) {
19         cerr << "Error: " << e.what() << endl;
20     }
21
22     return 0;
23 }
```


Output: example if a user enter 2

```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
Enter the numerator:2
Enter the denominator:2
Result: 1

Process finished with exit code 0
|
```

In the program:

- The user is prompted to enter a numerator and a denominator.
- Before performing the division, the program checks if the denominator is zero.
- If the denominator is zero, the program throws an exception of type `runtime_error`.
- The exception is caught in the catch block, and an error message is displayed to the user.
- If there's no exception, the division is performed, and the result is displayed.

Exercise 7.2: Extend Exercise 7.1 by adding exception handling for invalid input

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Activity: Extend Exercise 7.1 by adding exception handling for invalid input. If the user enters a non-numeric value, catch and handle the exception and display an appropriate error message.

LAB 8: Standard Template Library (STL)

Objectives:

- Learn about Sequence containers (e.g., vector, list)
- Get introduced to the vast collection of STL algorithms available for operations like searching, sorting, modifying, etc

Experiment 8.1: Introduction to STL Containers (Vectors, Lists)

- Write a program that uses a list to implement a to-do list. Allow the user to add tasks to the list and mark tasks as completed. Implement functions to add tasks, mark tasks as done, and display the list of tasks

Algorithm:

1. START
2. DEFINE a class named "Task" with attributes "description" (a string) and "completed" (a boolean).
3. WITHIN "Task" class:
 - a. CONSTRUCTOR: Initialize the "description" attribute with given value and set "completed" attribute to false.
 - b. FUNCTION "markCompleted":
 - Set "completed" to true.
 - c. FUNCTION "display" (should be a constant function since it doesn't modify any class attributes):
 - IF "completed" is true:
 1. OUTPUT "[DONE]" followed by "description".
 - ELSE:
 1. OUTPUT "[TODO]" followed by "description".
4. DEFINE a class named "ToDoList" containing a list of "Task" objects named "tasks".
5. WITHIN "ToDoList" class:
 - a. FUNCTION "addTask" accepting a string "desc":
 - ADD a new Task object with description "desc" to the "tasks" list.
 - b. FUNCTION "markTaskCompleted" accepting a string "desc":
 - FOR EACH "task" in "tasks":
 1. IF "task's description" matches "desc":

- a. Call "markCompleted" for that task.
 - b. RETURN from the function.
 - OUTPUT "Task not found!"
 - c. FUNCTION "displayTasks":
 - FOR EACH "task" in "tasks":
 1. Call the "display" function of that task.
6. MAIN FUNCTION:
- a. CREATE an object of "ToDoList" named "myToDoList".
 - b. DECLARE an integer variable "choice".
 - c. DO:
 - DISPLAY a menu with options to: Add Task, Mark Task as Completed, Display Tasks, and Exit.
 - INPUT user's choice into "choice".
 - CLEAR any possible invalid inputs from the input stream.
 - SWITCH based on the user's choice:
 1. OPTION 1:
 - a. DECLARE a string "desc".
 - b. PROMPT user for task description.
 - c. INPUT into "desc".
 - d. CALL "addTask" method of "myToDoList" with "desc".
 2. OPTION 2:
 - a. DECLARE a string "desc".

b. PROMPT user for task description to mark as completed.

c. INPUT into "desc".

d. CALL "markTaskCompleted" method of "myToDoList" with "desc".

3. OPTION 3:

a. CALL "displayTasks" method of "myToDoList".

4. OPTION 4:

a. DISPLAY "Goodbye!".

5. DEFAULT:

a. DISPLAY "Invalid choice! Please try again..".

- REPEAT while "choice" is not 4.

7. END

Code:

```
1  #include <iostream>
2  #include <list>
3  #include <string>
4  using namespace std;
5
6  class Task {
7  public:
8      string description;
9      bool completed;
10
11     Task(string desc) : description(desc), completed(false) {}
12
13     void markCompleted() {
14         completed = true;
15     }
16
17     void display() const {
18         cout << (completed ? "[DONE] " : "[TODO] ") << description << endl;
19     }
20 };
21
```

```

21
22 class TodoList {
23 private:
24     list<Task> tasks;
25
26 public:
27     void addTask(const string& desc) {
28         tasks.push_back(x: Task(desc));
29     }
30
31     void markTaskCompleted(const string& desc) {
32         for (Task& task : tasks) {
33             if (task.description == desc) {
34                 task.markCompleted();
35                 return;
36             }
37         }
38         cout << "Task not found!" << endl;
39     }

```

```

40
41     void displayTasks() {
42         for (const Task& task : tasks) {
43             task.display();
44         }
45     }
46 };
47
48 int main() {
49     TodoList myTodoList;
50     int choice;
51
52     do {
53         cout << "\nTo-Do List Menu:\n";
54         cout << "1. Add Task\n";
55         cout << "2. Mark Task as Completed\n";
56         cout << "3. Display Tasks\n";
57         cout << "4. Exit\n";
58         cout << "Enter your choice: ";
59         cin >> choice;
60
61         cin.ignore(); // Clear newline left in buffer
62

```

```
62
63     switch (choice) {
64     case 1: {
65         string desc;
66         cout << "Enter task description: ";
67         getline(&cin, &desc);
68         myToDoList.addTask(desc);
69         break;
70     }
71     case 2: {
72         string desc;
73         cout << "Enter task description to mark as completed: ";
74         getline(&cin, &desc);
75         myToDoList.markTaskCompleted(desc);
76         break;
77     }
78     case 3:
79         myToDoList.displayTasks();
80         break;
81
82         case 4:
83             cout << "Goodbye!\n";
84             break;
85         default:
86             cout << "Invalid choice! Please try again.\n";
87     }
88     } while (choice != 4);
89
90     return 0;
91 }
```


Output: example the user wants to add a task.

```
"C:\Users\Owner\CLionProjects\C program\Cpp program.exe"

To-Do List Menu:
1. Add Task
2. Mark Task as Completed
3. Display Tasks
4. Exit
Enter your choice:1
  Enter task description:Take C++ Lab practicals

To-Do List Menu:
1. Add Task
2. Mark Task as Completed
3. Display Tasks
4. Exit
Enter your choice:3
  [TODO] Take C++ Lab practicals
```

In this program:

- We define a Task class that contains a description and a completion status.
- We define a TodoList class that holds a list of tasks and provides methods to add tasks, mark tasks as completed, and display tasks.
- In the main function, we present a menu for the user to interact with their to-do list.
- The user can add tasks, mark tasks as completed, display the list of tasks, and exit the program.

Exercise 8.1: Create a program that uses a vector to maintain a list of student names. Allow the user to add new students to the vector and display the list of students. Implement a function to remove a student by name from the vector.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Experiment 8.2: STL Algorithms (Sorting, Searching)

- **Write a simple C++ program that demonstrates the use of generic function template that finds the minimum value in an array for different data types**

Algorithm:

1. START
2. DEFINE a generic function template "findMin" that accepts an array "arr" of type T and its size.
3. WITHIN "findMin":
 - a. IF size is 0:
 - i. RETURN default value of type T.
 - b. SET "minVal" to the first element of "arr".
 - c. FOR each element "elem" in "arr" starting from the second element:
 - i. IF "elem" is less than "minVal":

1. UPDATE "minVal" to "elem".

d. RETURN "minVal".

4. MAIN FUNCTION:

a. DECLARE an integer array "intArr" with values: 3, 1, 4, 1, 5, 9, 2, 6, 5, 3.

b. DECLARE a double array "doubleArr" with values: 3.14, 1.59, 2.65, 3.58, 9.79.

c. DISPLAY "Minimum value in intArr: " followed by the result of "findMin" called with "intArr" and its size.

d. DISPLAY "Minimum value in doubleArr: " followed by the result of "findMin" called with "doubleArr" and its size.

5. END

Code:

```

1  #include <iostream>
2  using namespace std;
3
4  // Generic function template to find the minimum value in an array
5  template <typename T>
6  T findMin(T arr[], int size) {
7      if (size == 0) {
8          return T(); // Default return value if the array is empty
9      }
10
11     T minVal = arr[0];
12     for (int i = 1; i < size; i++) {
13         if (arr[i] < minVal) {
14             minVal = arr[i];
15         }
16     }
17     return minVal;
18 }
19

```

```

20 int main() {
21     int intArr[] = { [0]: 3, [1]: 1, [2]: 4, [3]: 1, [4]: 5, [5]: 9, [6]: 2, [7]: 6, [8]: 5, [9]: 3};
22     double doubleArr[] = { [0]: 3.14, [1]: 1.59, [2]: 2.65, [3]: 3.58, [4]: 9.79};
23
24     cout << "Minimum value in intArr: " << findMin(arr: intArr, size: sizeof(intArr) / sizeof(int)) << endl;
25     cout << "Minimum value in doubleArr: " << findMin(arr: doubleArr, size: sizeof(doubleArr) / sizeof(double)) << endl;
26
27     return 0;
28 }
29

```

Output:

```

↑ "C:\Users\Owner\CLionProjects\C program\Cpp program.exe"
↓ Minimum value in intArr: 1
↶ Minimum value in doubleArr: 1.59
↷ Process finished with exit code 0
|
|
|
|

```

In the above program:

- A generic function template `findMin` is defined. This function calculates the minimum value in an array for different data types (T).
- Inside the main function, two arrays are defined (`intArr` and `doubleArr`) of different data types (int and double, respectively).
- The `findMin` function is then called for each array, and the results are displayed.

Exercise 8.2: Write a program that uses the STL algorithm `binary_search` to search for a specific element in a sorted vector of integers. Prompt the user to enter a number, populate and sort the vector, and then search for the entered number using `binary_search`. Display whether the number was found or not.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs

Activity:

LAB 9: Final Project (CA) - 2 weeks

Project Title: Library Management System

Project Description:

You are tasked with creating a C++ program for a **Library Management System** that allows users to manage books and patrons. The program should provide the following functionality:

1. Create a new C++ project in a development environment (e.g., JetBrains Clion).
2. Write a program that displays a welcome message and allows the user to enter their name. The program should then greet the user with their name.
3. Implement a menu-based system that allows users to perform the following actions:
 - Add a new book.
 - Display the list of all books.
 - Search for a book by title or author.
 - Add a new patron.
 - Display the list of all patrons.
 - Check out a book to a patron.
 - Return a book from a patron.
 - Display the list of books checked out by a patron.
 - Exit the program.
4. Use arrays or vectors to store book and patron objects. You should be able to add, remove, and modify book and patron records within these arrays or vectors.

5. Create a class representing a Book with attributes like title, author, ISBN, and availability status (checked out or available). Instantiate and display book objects.
6. Create a class representing a Patron with attributes like name, library card number, and a list of books checked out. Instantiate and display patron objects.
7. Implement methods in the Patron class to check out and return books.
8. Use dynamic memory allocation to create arrays of book and patron objects. Write a program that allows the user to input information for multiple books and patrons and then displays the details of all books and patrons.
9. Implement file handling to save book and patron records to separate text files and load records from these files. Ensure that you handle file open and close operations gracefully.
10. Implement exception handling to catch and handle any exceptions that may occur during file operations, such as file not found or file access errors.
11. Use the STL vector container to maintain a list of book titles and author names. Allow the user to add new books to the vector and display the list of books. Implement a function to remove a book by title from the vector.

Project Submission: Submit your C++ program along with a brief documentation explaining how to use it. Ensure your code is well-organized, and you have used appropriate comments and function headers.

Note: This project encompasses the fundamental and advanced concepts of C++ programming, including variables, control structures, object-oriented programming, file handling, exception handling, and the use of the STL, as shown in the previous labs. It provides a comprehensive opportunity for learners to apply what they've learned in the previous labs.

Notes

1. A very good manual
2. There should be a cover page
3. Modify as suggested
4. Provide enough spaces between sentences and screenshots

Edit and revert to me for final approval